HCCL集合通信专用引擎CCU技术介绍

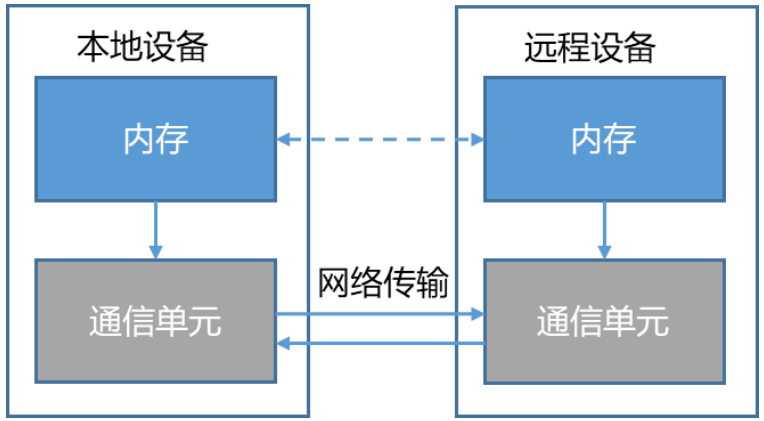
作者：陈仙萍 文学敏

时间：2026/3

CCU设计背景及初衷 – IO带宽对于内存带宽的挑战

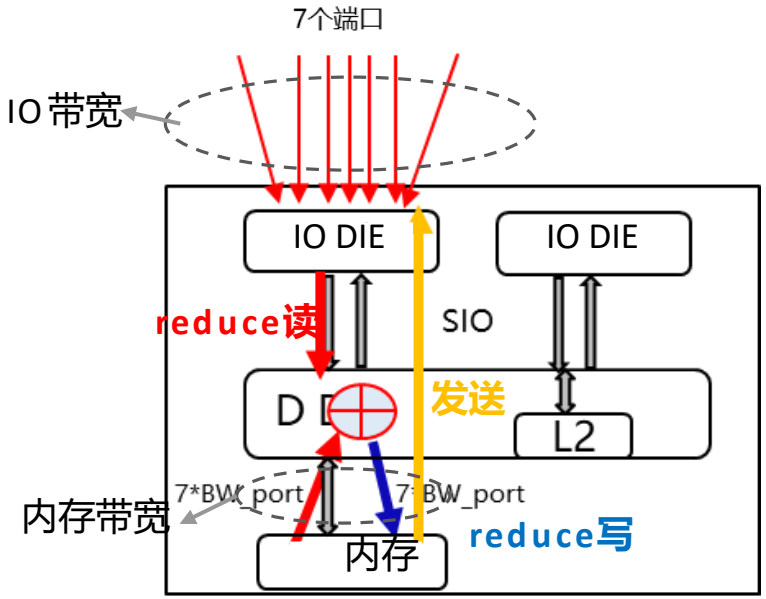
AI通信算子流量特征

- **强突发性**：多卡同步时（AllReduce、Broadcast 等）瞬间产生大量数据读写，集中占用内存带宽与总线。
- **传输数据量大**：梯度聚合、参数同步、激活值传输都是大张量操作，单次通信数据量巨大。
- **高内存带宽依赖**：通信数据的传输、规约都要读写内存。



Ascend 950 Allreduce内存带宽

- **接收方向**：7个端口同时接收来自fullmesh对端的数据(size= M)，并排队求和，需7次读写内存，假设每个端口单向带宽为BW_port，则内存带宽需求为2*7* BW_port
- **发送方向**：本端数据从7个端口同时发送(size= M)，需7次读内存，内存带宽需求为7* BW_port
- **合计内存带宽需求**： 3*7* BW_port



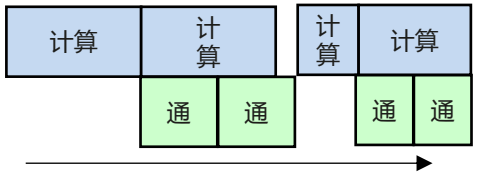
CCU设计背景及初衷 – 通信和计算算子抢占硬件资源

传统集合通信依赖 AI CPU、AI Vector 等计算核通过协议栈软件接口构造通信任务描述符，驱动硬件执行通信任务，这种执行需要占用计算核、软件接口的开销较大。叠加计算算子以后，计算和通信抢占多种硬件资源。独立的通信引擎可以解决该问题。

20年~ 22年
常规模型训练/推理方案

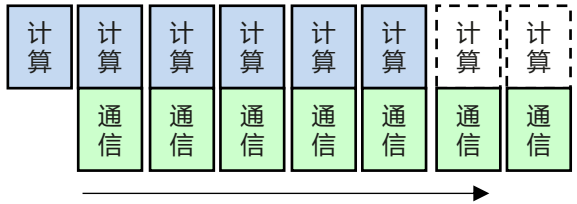
效果：计算和通信串行执行
适用场景：模型并行/序列并行/专家并行
问题：
- 计算和通信无法掩盖

23年-24年
粗粒度通算overlap



效果：计算和通信一部分overlap
适用场景：大数据量的计算/通信
问题：
- 访存带宽需求高

25年
细粒度通算overlap

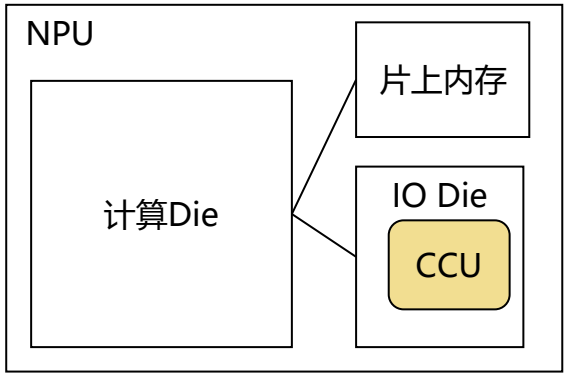


效果：计算& 通信融合并发执行
适用场景：模型并行/序列并行/专家并行/上下文并行
问题：
- 计算& 通信抢计算核资源
- 计算& 通信抢访存带宽

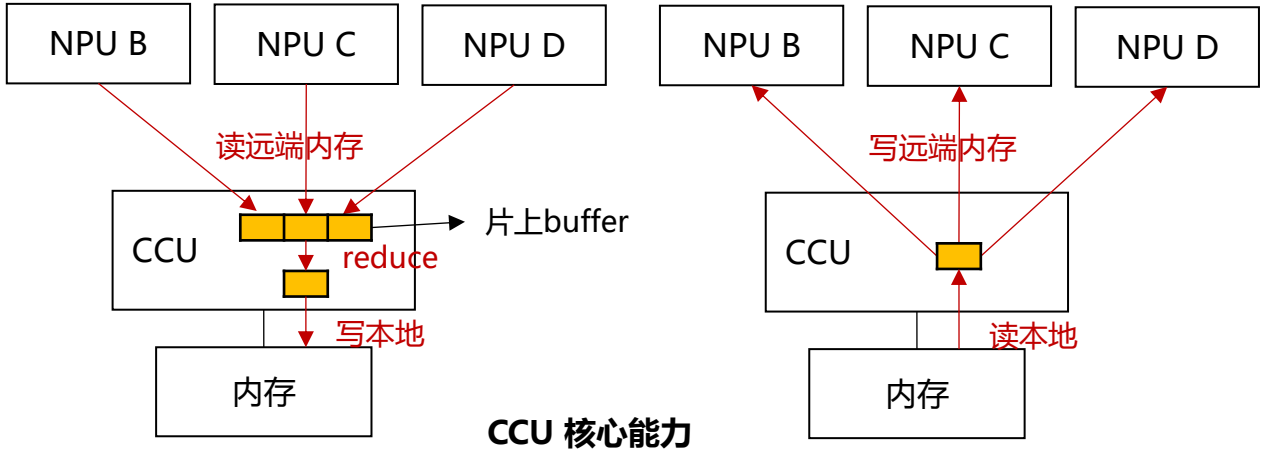
CANN

Ascend 950 专用通信引擎CCU – 降低内存带宽占用、降低通信时延、天然零拷贝及确定性计算

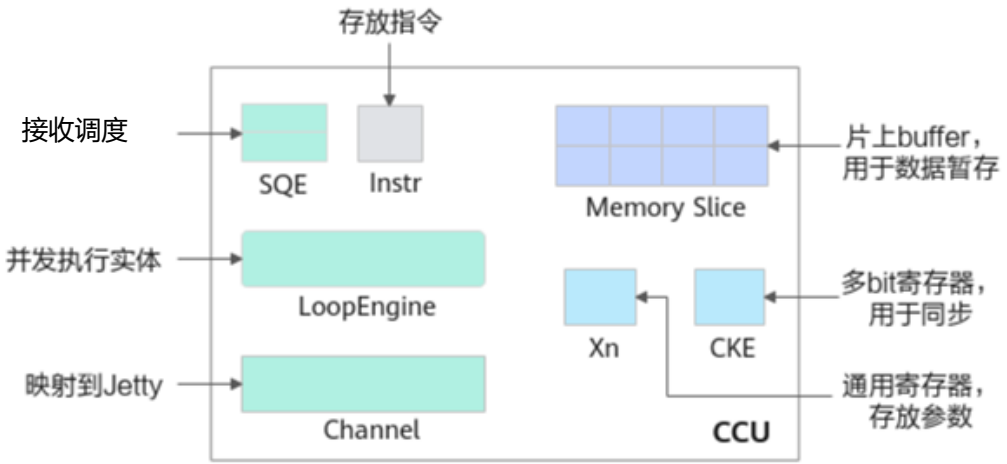
Collective Communication Unit（CCU）是Ascend 950上新增的用于加速集合通信的专用引擎，CCU根据软件预置的集合通信算法指令，完成通信节点间的同步、地址交换、数据传输以及规约计算，实现集合通信操作。



CCU位置示意图



CCU 核心能力



CCU 硬件逻辑架构

解决的问题及优势：

1、利用CCU片上buffer缓存通信数据，降低访存带宽需求；

2、利用片上buffer+内置计算单元执行Reduce操作，确保计算保序；

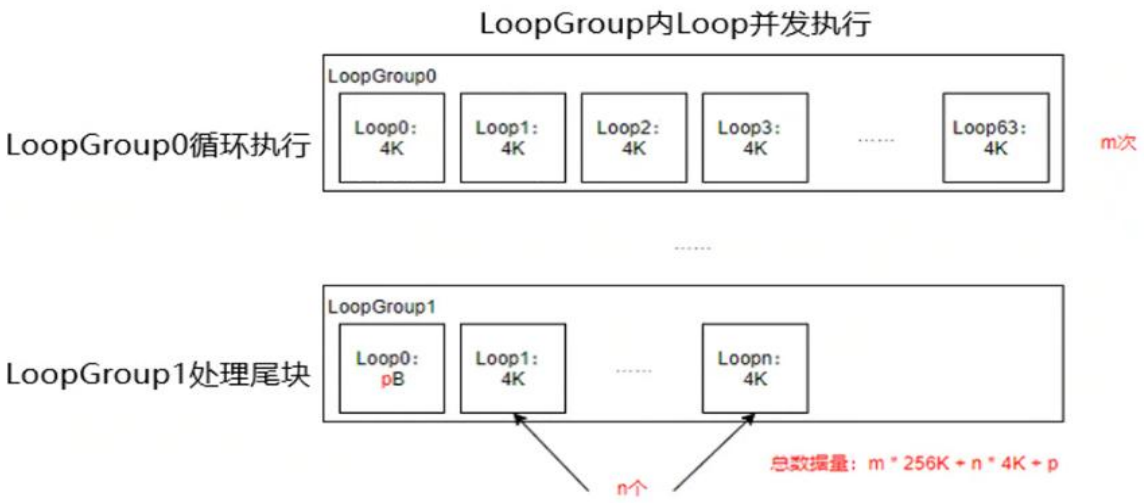
3、专用的通信调度与同步机制，降低时延，不占用计算算力；

4、消减用户buffer到CCL Buffer的拷贝

CANN

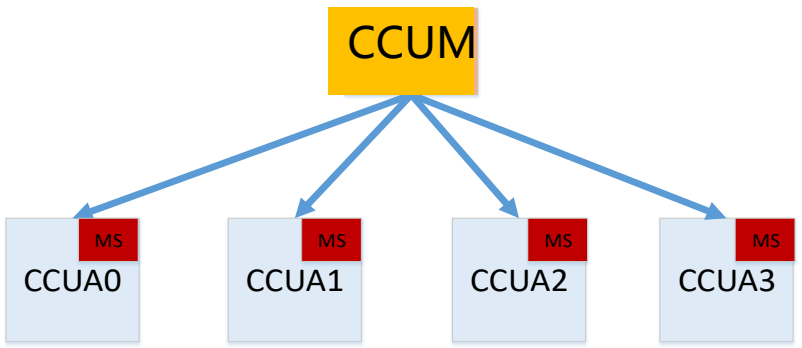
Ascend 950 CCU硬件能力---并发执行单元和片上内存

- CCU数据搬移过程由LoopEngine完成，每个Loop在MS上读写数据，多个Loop组合成一个Loop Group；
- 设计目的：每个Loop Group内的各个Loop是并发的，每个Loop需要等待自己的ack，依靠多个Loop依次发送来打满带宽，掩盖各个loop等待ack的时间；
- Loop Group之间是串行的，每个Loop Group内通过调整循环次数(m)、并发度(n)、搬运数据量(p)来对应不同的数据搬运量。



CCU并发执行单元示意图

- CCU由CCUM（CCU Manager）和CCUA（CCU Agent）单元构成，CCUM是CCU中的主要控制单元，CCUA是CCU中的主要数据处理单元，完成内存和片上缓存之间的数据传输，以及片上缓存间的数据归约
- Memory Slice用作数据buffer，位于CCUA上，Reduce与Broadcast输入输出数据都要经过MS；
- 所有MS分布在多个CCUA上以交织的方式编址



CCU组件示意图

Ascend 950 CCU硬件能力---片上资源及指令

CCU资源

- Memory Slice（MS）：用于缓存待传输或待规约的数据；
- LoopEngine：指令执行的硬件单元，可并发，LoopEngine支持根据循环控制信息重复执行；
- 地址寄存器Gather Scatter Address（GSA）：用于存储内存地址；
- 通用寄存器General Perpose Register（Xn）：用于存储Token信息、循环控制信息等；
- 同步寄存器Checklist Entry（CKE）：用于指令间的依赖关系表达；
- Channel表项：用于表达传输路径；

CCU指令

- 加载指令：将立即数或任务描述符中的字段加载到地址寄存器或通用寄存器中。
- 运算指令：实现地址寄存器与通用寄存器之间的加法运算，用于地址计算。
- 控制指令：包括条件跳转、Loop循环控制、LoopGroup并发控制等，用于控制CCU的执行流程。
- 同步指令：实现对Xn/GSA/CKE寄存器的跨CCU置位操作，以及本地CKE的置位/清零/等待操作，实现跨CCU的寄存器同步。
- 传输指令：实现本端/远端内存与MS之间的数据传输，支持随路归约。
- 归约指令：实现本地多片MS之间的数据归约，支持多种数据类型和归约类型（如加法、最大值、最小值等）。

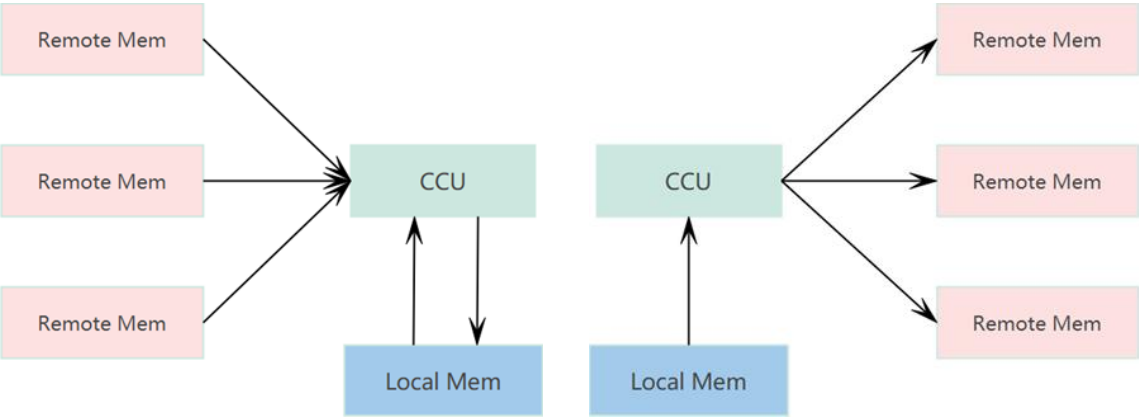
CANN

Ascend 950 CCU关键技术 – 基础指令封装、低时延调度

关键技术1：基于CCU集合通信基础指令封装

①对通信相关的数据类型进行抽象，对同步、数据搬运等操作进行封装

②提供CCU扩展指令：GroupBroadcast、GroupReduce等

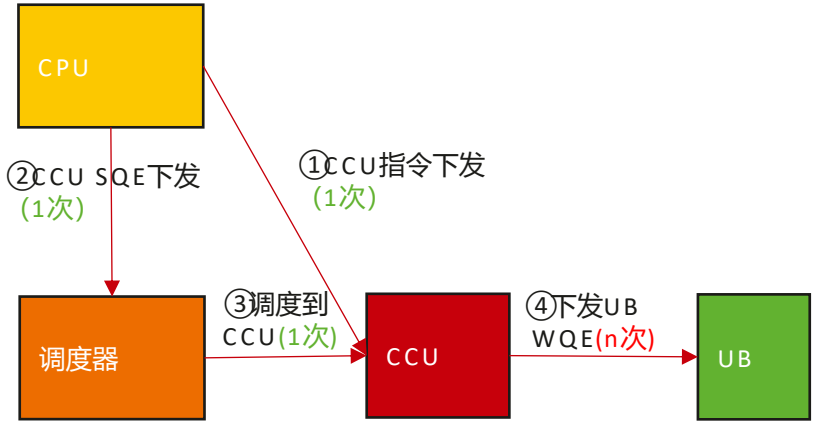


GroupReduce操作　　GroupBroadcast操作

关键技术2：基于CCU的集合通信软件调度

①通过SQE调度CCU，CCU再多次调度UB引擎

②CCU O(10ns)调度，解决小包调度时延大的问题



典型扩展指令序列

GroupReduce:

TransLocMemToLocMS：Read From Rank0 Mem to Loc MS0

TransRmtMemToLocMS：Read From Rank1 Mem to Loc MS1

TransRmtMemToLocMS：Read From Rank2 Mem to Loc MS2

TransRmtMemToLocMS：Read From Rank3 Mem to Loc MS3

AddFP16：Reduce MS0 ~ MS3 to MS 0

TransLocMSToLocMem：Write MS0 to Rank0 Mem

GroupBroadcast:

TransLocMemToLocMS：Read From Rank0 Mem to Loc MS0

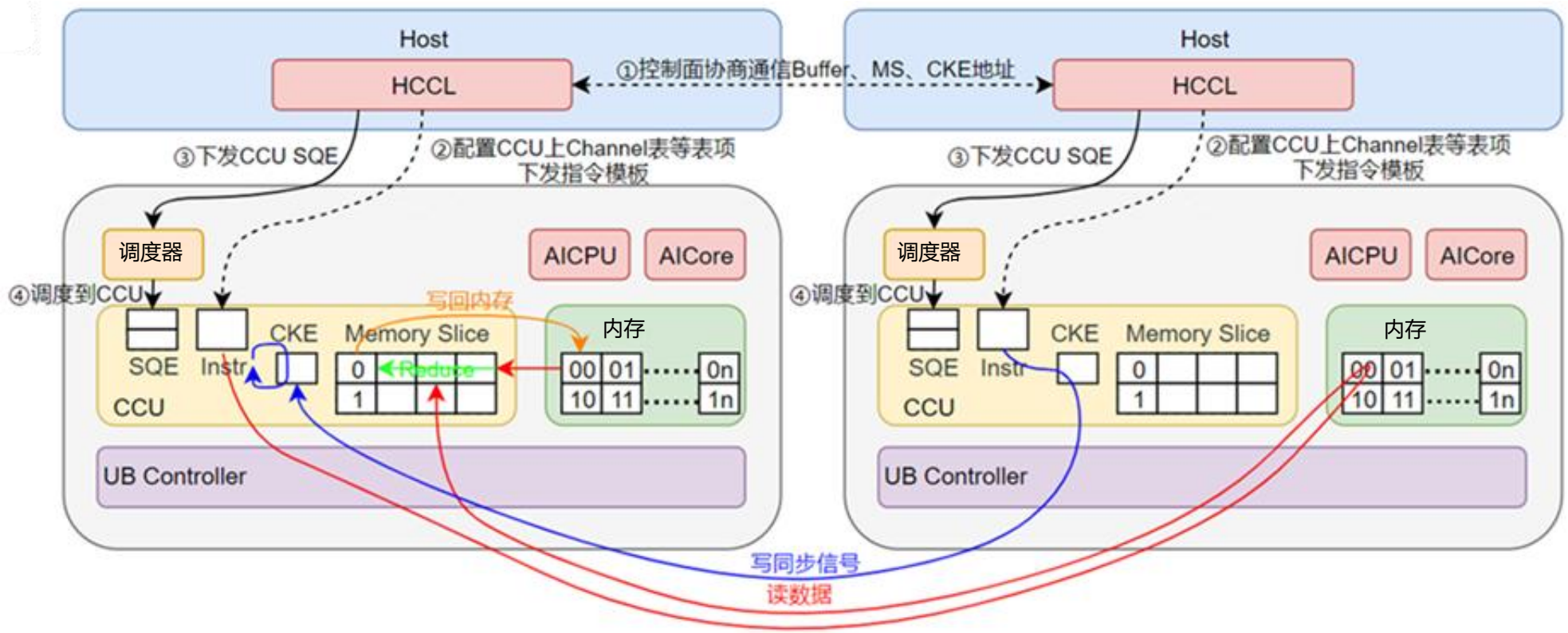
TransLocMSToRmtMem：Write Loc MS0 to Rank1 Mem

TransLocMSToRmtMem：Write Loc MS0 to Rank2 Mem

TransLocMSToRmtMem：Write Loc MS0 to Rank3 Mem

Ascend 950 CCU关键技术 – 通信过程硬化执行，节省调度开销，释放核资源

软件构造集合通信指令序列，控制面流程打通，通信过程硬化执行



CANN

https://gitcode.com/cann

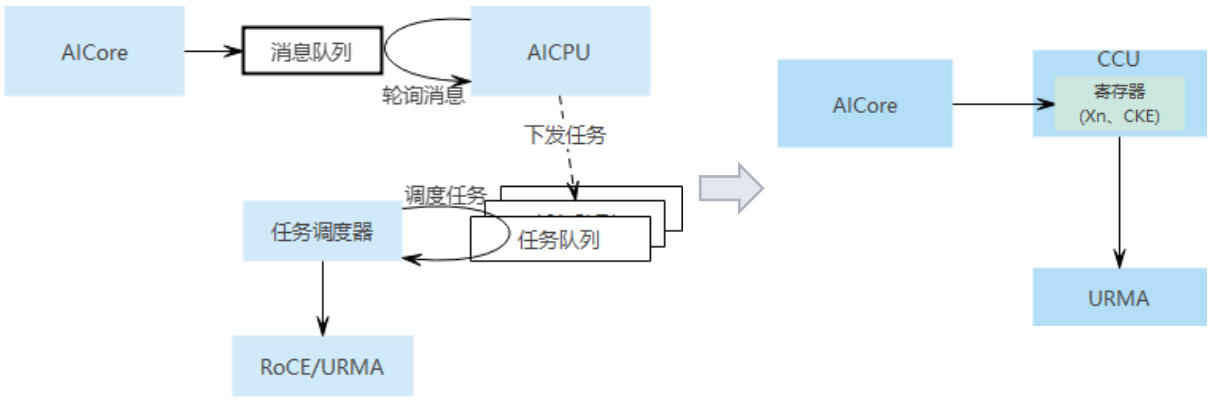
Ascend 950 CCU关键技术 – AI Core & CCU高效协同，节省核资源

基于CCU的集合通信通算融合，AIV 和 CCU高效协同

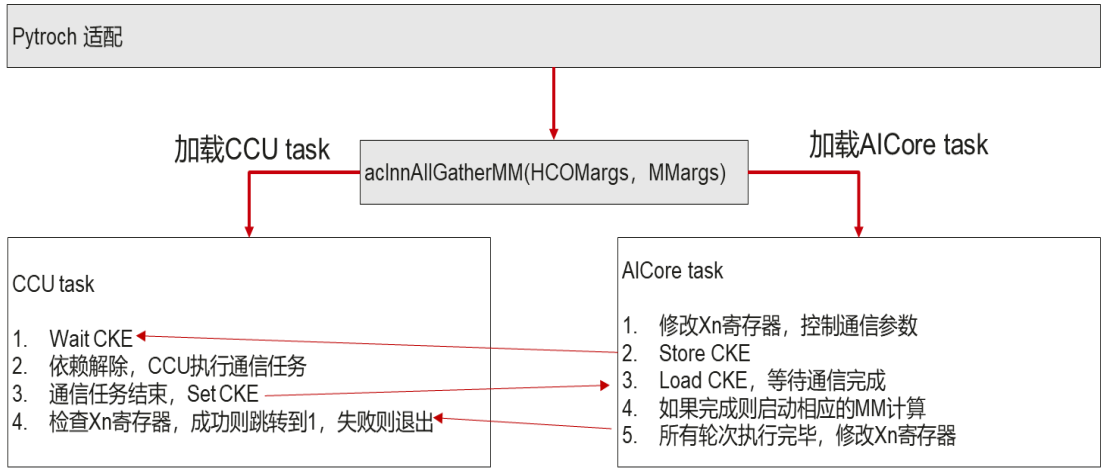
①通信任务由CCU发起，相比原来AICPU发起的模式，通信任务展开更快，调度更高效

②AI core和通信引擎之间通过寄存器传递同步及参数信息，交互路径短、时延低，从发起任务到通信执行时延小

Ascend 950通算融合变化



计算和通信协同流程



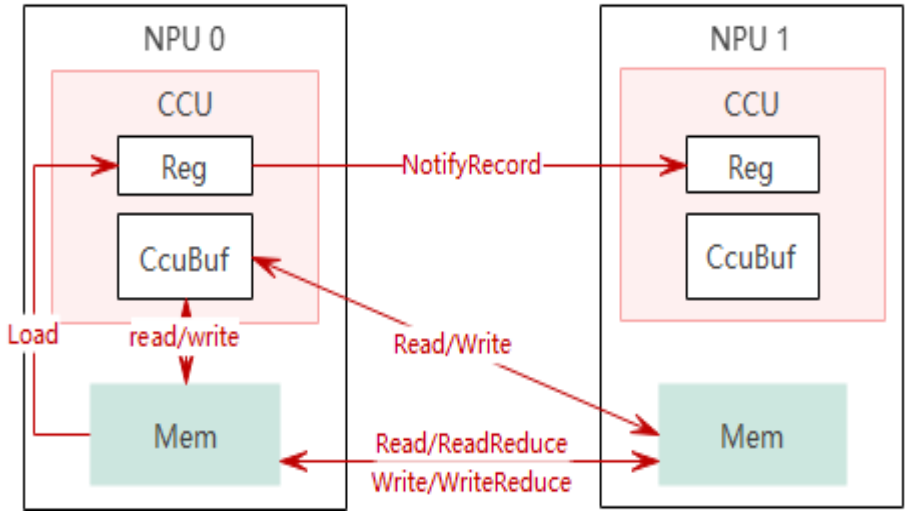
无AICPU参与，AICore和CCU之间直接交互，信息传递路径AICore -> CCU

优势：传递路径短，时延低

劣势：AICore需要感知到CCU内部逻辑

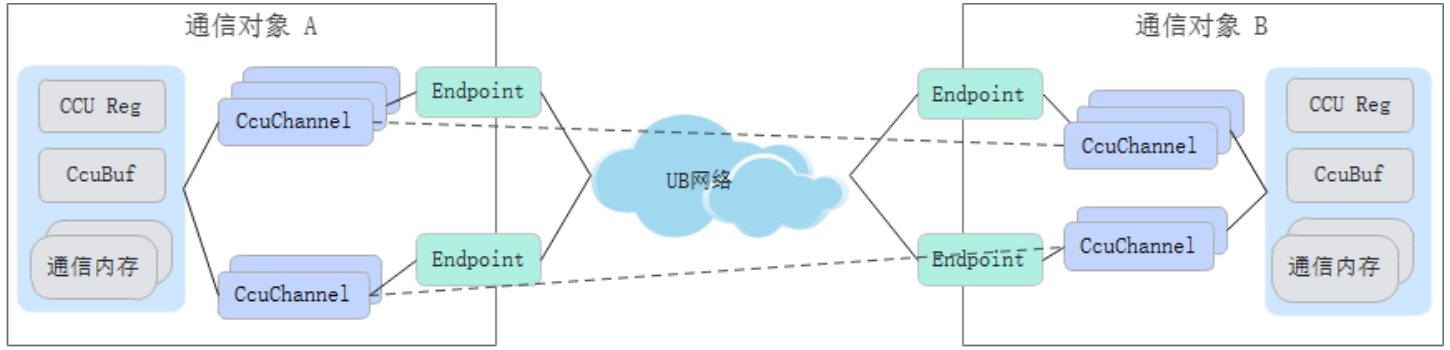
CCU编程模型 – 数据搬运

CCU数据搬运能力

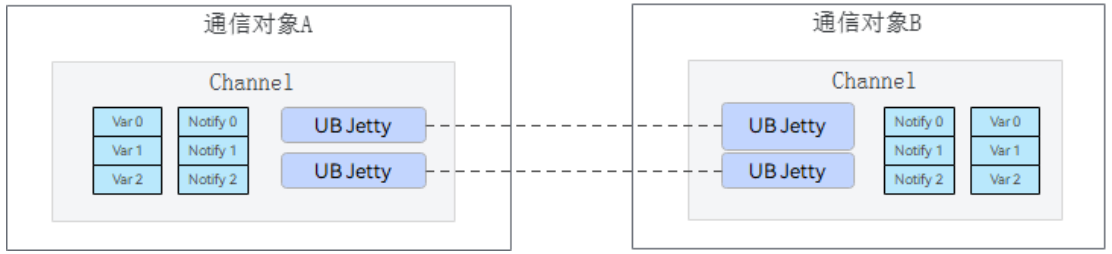


- Local CcuBuf <-> Local Mem
- Local CcuBuf <-> Remote Mem
- Local Mem <-> Remote Mem
- Local CcuVariable -> Remote CcuVariable

CCU通信模型



- Endpoint：通信对象的数据收发端口；
- CcuChannel：不同通信对象的Endpoint之间建立的通信通道；
- 开发者使用Channel可以读写远端通信对象内存或与远端通信对象进行同步。



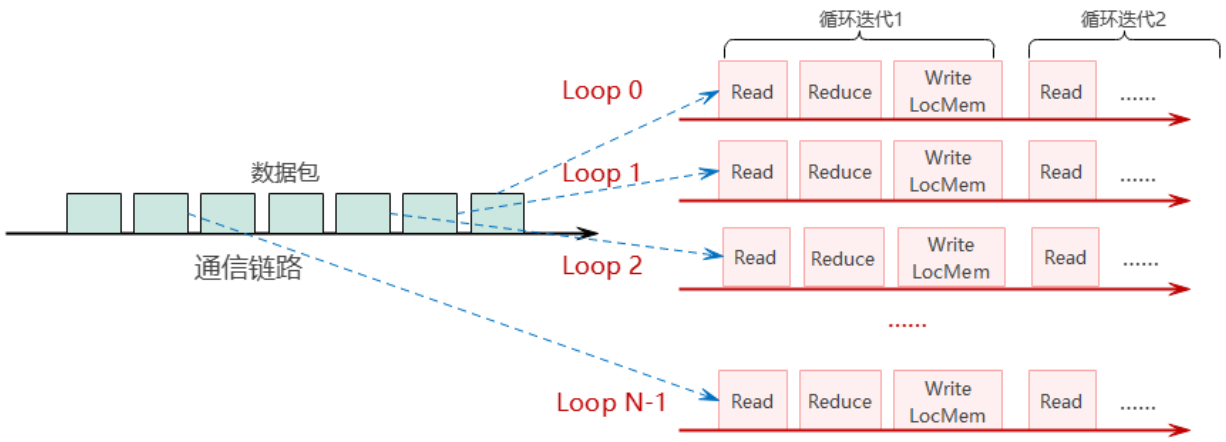
CcuChannel模型

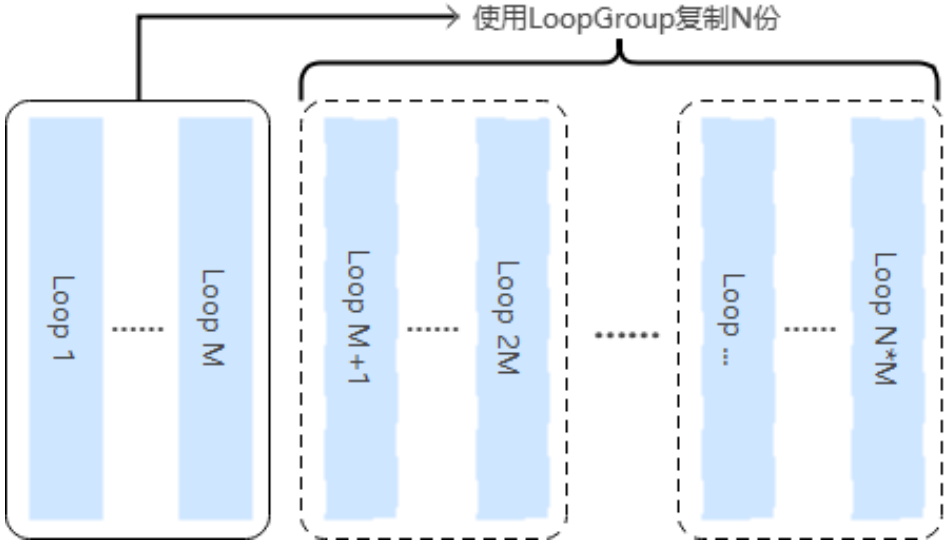
CANN

CCU编程模型 – 并发

CCU并发模型

并发搬运数据，减少头开销，提升带宽利用率



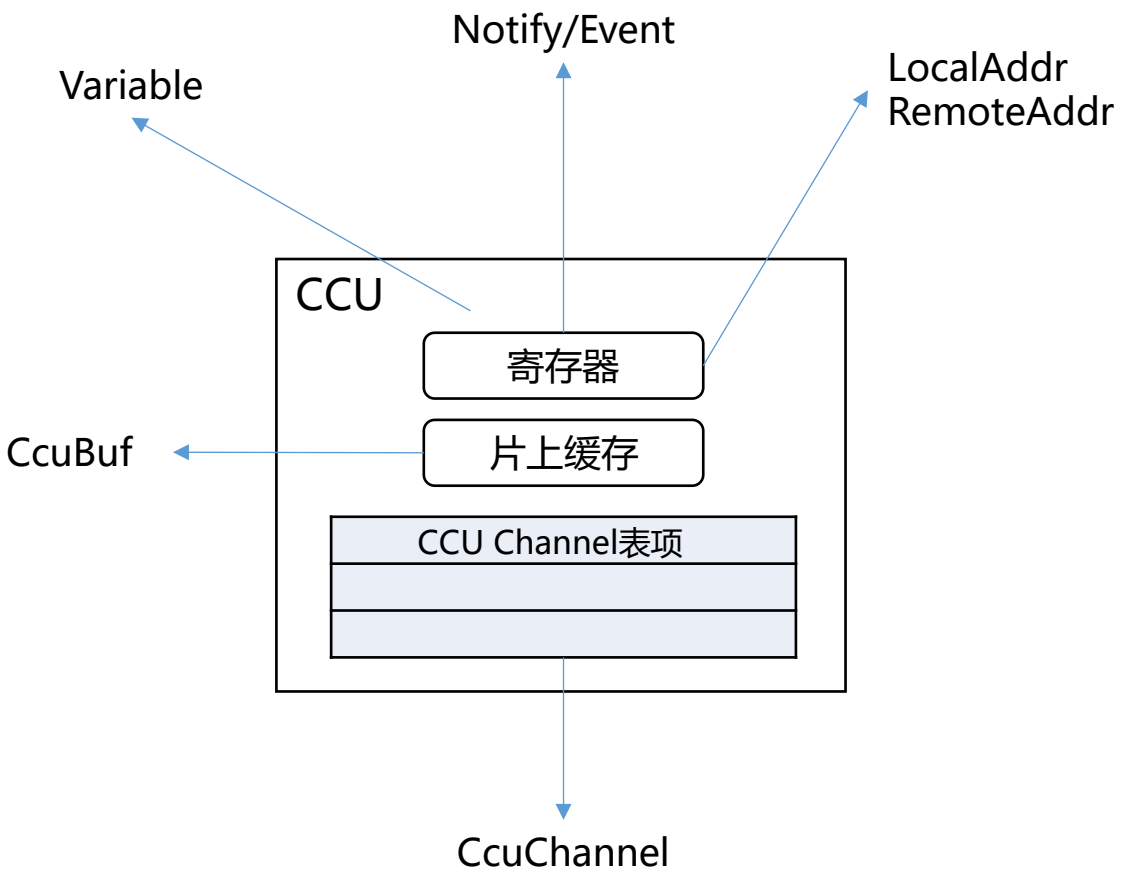


Loop：
- 一个CCU任务中的基础并发单元；
- Loop内的操作串行，Loop间并行；
- 配合CcuBuf循环搬运数据；
- Loop参数：循环次数、每次循环操作的内存地址增量偏移；

LoopGroup：
- 将一组Loop复制多份，增加并发Loop个数；
- 避免重复的指令逻辑手写多份。

CCU编程模型 – 数据类型与接口

CCU数据类型

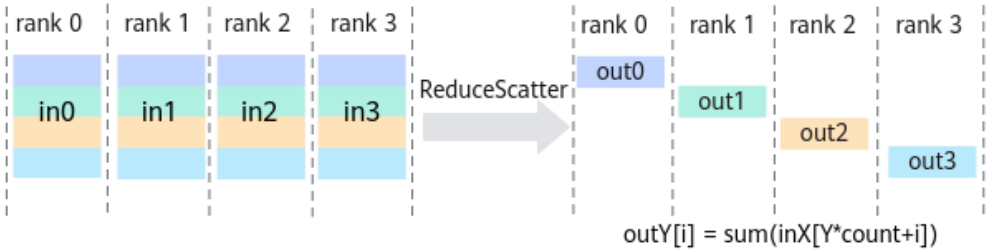


CCU编程接口示意

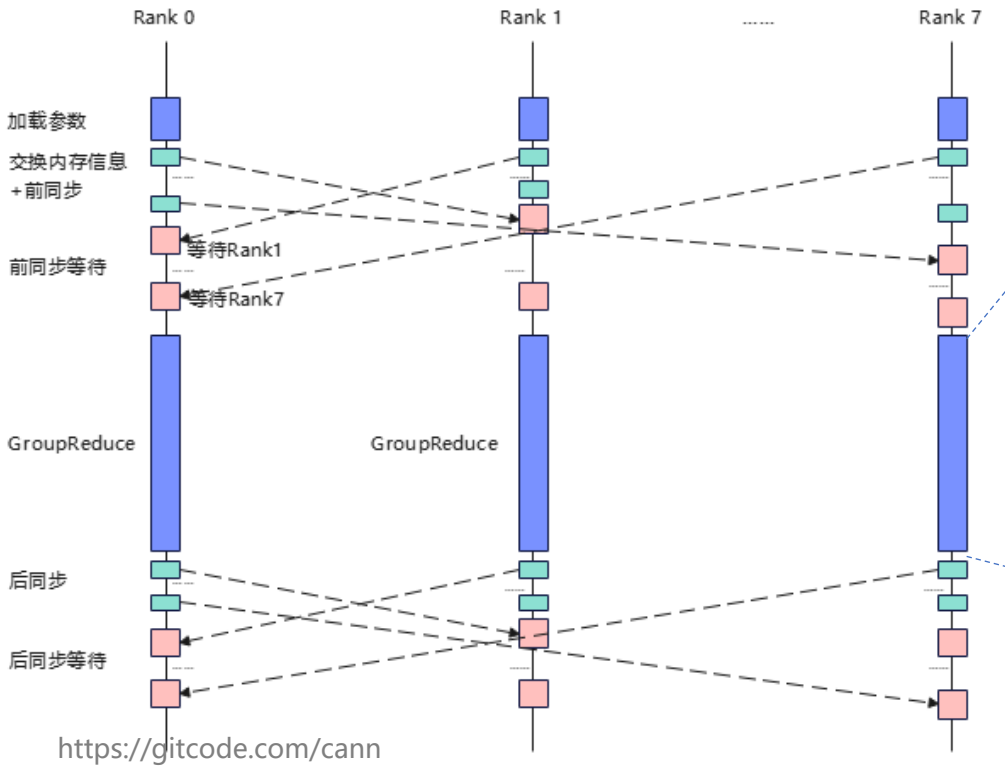
	接口示意
同步操作	NotifyRecord(notify)
	NotifyWait(notify)
	ChannelRecord(channel, notifyIdx)
	WriteWithNotify(channel, var, varIdx, notifyIdx)
	ChannelWait(channel, notifyIdx)
远端操作	Write(channel, remMem, locMem, len, completedEvent)
	Write(channel, rem, LocalBuf loc, len, completedEvent)
	Read(channel, loc, rem, len, completedEvent)
	Read(channel, LocalBuf loc, rem, len, completedEvent)
	WriteReduce(channel, rem, loc, len, dataType, redopType, completedEvent)
	ReadReduce(channel, loc, rem, len, dataType, redopType, completedEvent)
本地操作	LocalCopy(dst, src, len, completedEvent)
	LocalCopy(localBuf dst, src, len, completedEvent)
	LocalCopy(dst, localBuf src, len, completedEvent)
	LocalReduce(dst, src, len, dataType, redopType, completedEvent)
	LocalReduce(bufs, bufCount, count, dataType, outputDataType, redopType, completedEvent)
参数加载	Load(var)、LoadArgs(var, argIdx)
控制语句	CCU_IF、CCU_WHILE
	LoopCall Loop(label)
	LoopGroup(loops, loopCfgs, loopCount, paraCfg, offsetCfg)
	Func(label)

CCU的使用与开发 – 8P Fullmesh场景的ReduceScatter算子

ReduceScatter算子



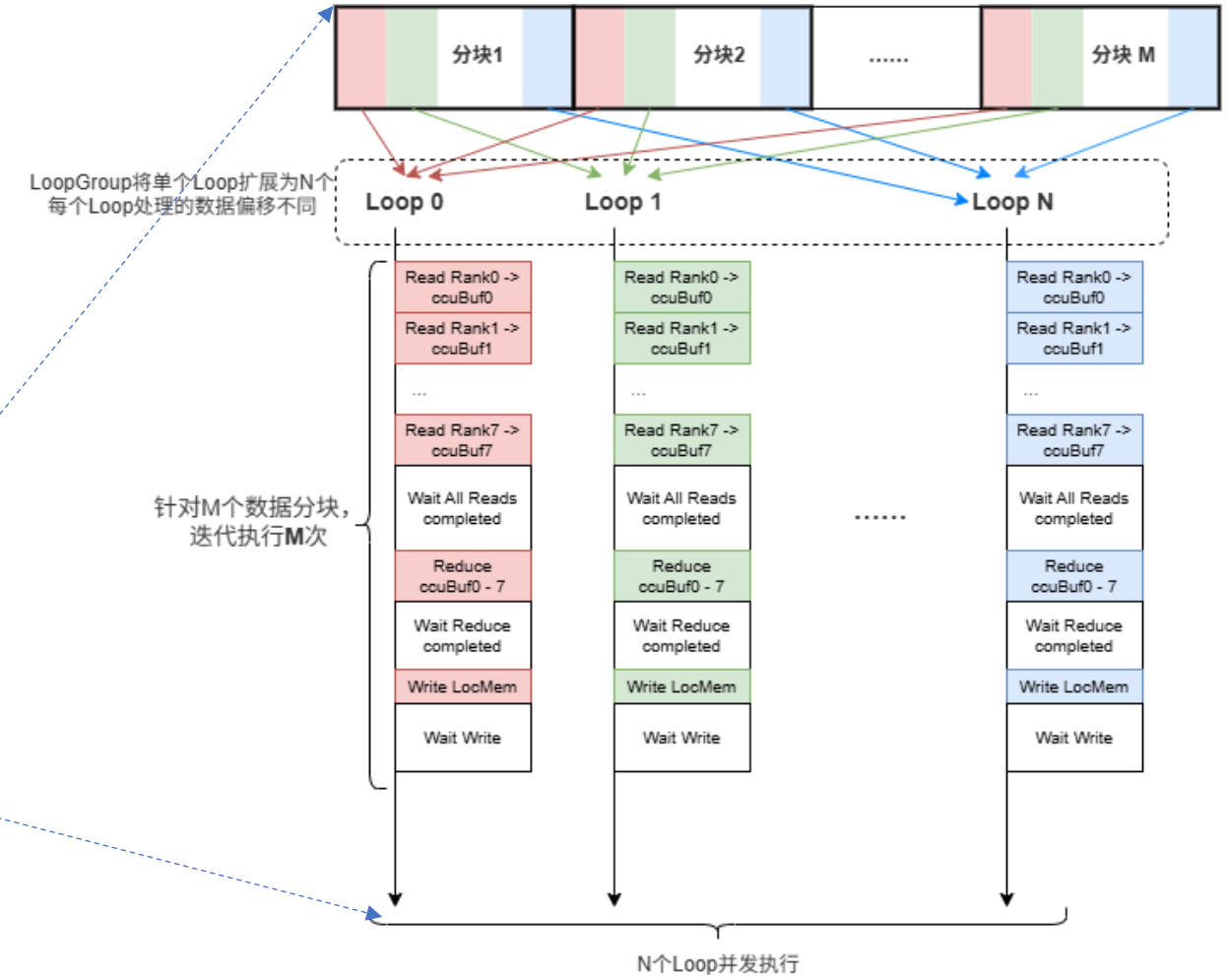
CCU实现ReduceScatter算子整体流程



https://gitcode.com/cann

CCU实现ReduceScatter的关键数据搬运流程

M：M个数据块，每个Loop需要循环迭代M次
N：自动复制的Loop数，每个Loop处理对应颜色的分片



如果不使用CcuBuf，使用Local Mem -> Remote Mem能力，也可以实现算子功能
性能收益主要来自于CCU的高效调度能力

CCU的使用与开发 – 8P Fullmesh场景的ReduceScatter算子

代码示意

```
{
    // 准备寄存器，加载参数
    CcuVariable inputs[rankSize], output;
    ...
    // 交换地址信息+ 前同步
    for (auto ch : channels) {
        NotifyRecord(ch, EVENT_0, INPUT_VAR_ID, inputs[myRank], 1 << INPUT_VAR_ID);
        NotifyRecord(ch, EVENT_0, TOKEN_VAR_ID, token_[myRank], 1 << TOKEN_VAR_ID);
    }
    uint32_t allBit = 1 << INPUT_VAR_ID  |1 << TOKEN_VAR_ID;
    for (auto ch : channels_) {
        NotifyWait(ch, EVENT_0, allBit);
    }

    // 准备做groupReduce需要的参数
    RemoteAddr src[rankSize];
    LocalAddr dst = CreateLocalAddr(output, token);
    for (uint32_t rank = 0; rank < rankSize; rank++) {
        src[rank] = CreateLocalAddr(inputs[rank], tokens[rank]);
    }
    GroupReduce(channels, dst, src, ..., dataType, outputDataType, reduceOp);

    // 后同步
    for (auto ch : channels) {
        NotifyRecord(ch, EVENT_0, 1 << POST_SYNC_ID);
    }
    for (auto ch : channels) {
        NotifyWait(ch, EVENT_0, 1 << POST_SYNC_ID);
    }
}
```

准备寄存器，并从任务描述符中加载参数

与其他Rank交换地址信息+ 前同步，等待所有Rank都准备好

读取其他Rank数据并执行Reduce

后同步，等待其他Rank都执行完成

CANN

Ascend 950实测集合通信性能：8P fullmesh拓扑，性能符合预期

Ascend 950集合通信竞争力主要来源于以下几方面：

1） **互联的物理带宽增加；**

2） **UB灵衢互联带宽利用率优化；**

3） **CCU使能；**

4） **创新的集合通信算法充分发挥带宽优势；**

集合通信性能	测试场景	Ascend 950相比上一代通信性能提升比		性能收益来源
		理论提升比	实测提升比	
Allgather算子	8KB@8P性能	114%	110%	(3)
	1GB@8P性能	41%	32%	(1), (2), (4)
Reducescatter	8KB@8P性能	114%	67%	(3)
	1GB@8P性能	74%	61%	(1), (2), (4)
AllReduce算子	8KB@8P性能	121%	63%	(3)
	1GB@8P性能	41%	40%	(1), (2), (4)
All2All算子	64KB@8P性能	112%	107%	(3),
	1GB@8P性能	146%	138%	(1), (2), (3), (4)

CANN

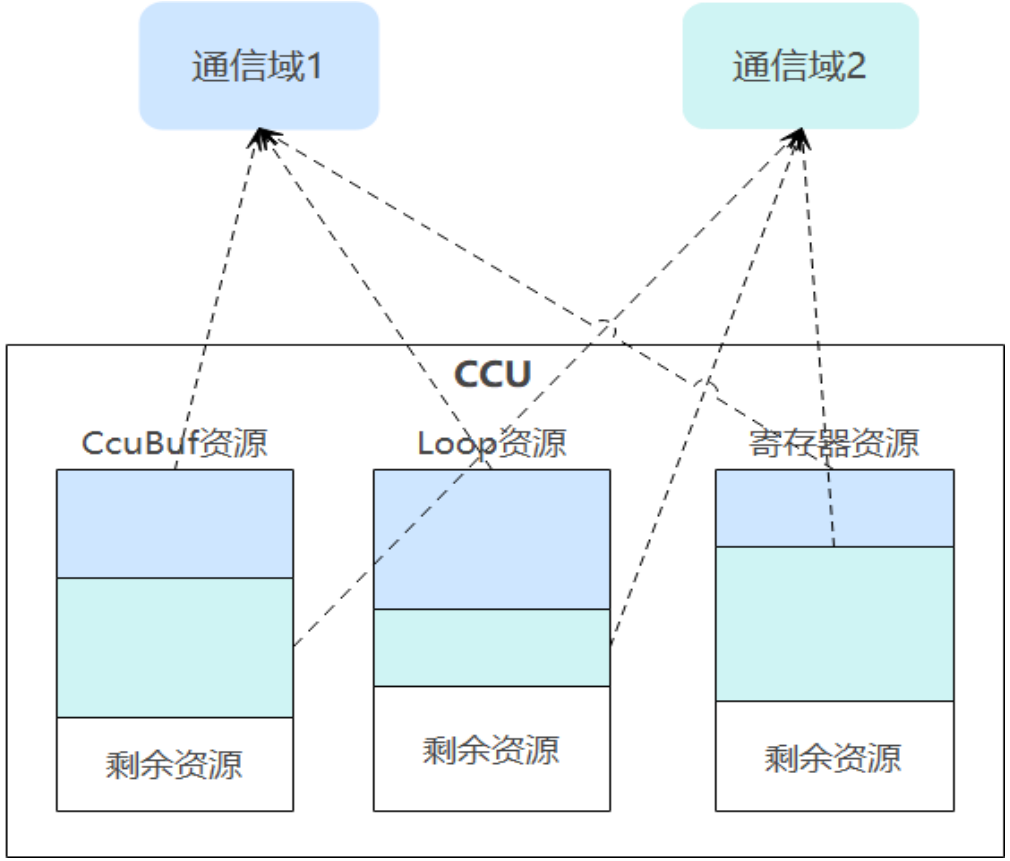
CCU硬化资源— 并发执行引擎、片上内存MS资源、channel资源

以下规格影响CCU支持的通信实例数及支持的通信域大小

硬化资源1：CCU并发执行单元loop实现并发数据传输，共200个loop单元

硬化资源2：CCU片上MS，即片上数据缓存单元，实现reduce和Broadcast基础能力，MS总大小6MB

硬化资源3：CCU硬化通信通道channel表项，该表记录CCU可以连接的对端数和相关的通信上下文信息，该表共128个record



Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann